# DS 3 : Gestion des processus, Récursivité

## Exercice 1 (7 points)

Cet exercice porte sur la programmation Python, la programmation orientée objet, les structures de données (file), l'ordonnancement et l'interblocage.

On s'intéresse aux processus et à leur ordonnancement au sein d'un système d'exploitation. On considère ici qu'on utilise un monoprocesseur.

1. Citer les trois états dans lesquels un processus peut se trouver.

   **Correction**
   Les états possibles sont prêt, élu et bloqué.

   On veut simuler cet ordonnancement avec des objets. Pour ce faire, on dispose déjà de la classe `Processus` dont voici la documentation :

   **Classe Processus :**

   - p = Processus(nom : str, duree : int)
     Crée un processus de nom `nom` et de durée `duree` (exprimée en cycles d'ordonnancement)
   - p.execute_un_cycle()
     Exécute le processus donné pendant un cycle.
   - p.est_fini()
     Renvoie True si le processus est terminé, False sinon.

   Pour simplifier, on ne s'intéresse pas aux ressources qu'un processus pourrait acquérir ou libérer.

2. Citer les deux seuls états possibles pour un processus dans ce contexte.

   **Correction**
   Pas de ressource à récupérer donc pas d'état "bloqué". Les états possibles sont donc prêt et élu.

   Pour mettre en place l'ordonnancement, on décide d'utiliser une file, instance de la classe `File` ci-dessous.

```
class File:
        def __init__(self):
                """ Cree une file vide """
                self.contenu = []

        def enfile(self, element):
                """ Enfile element dans la file """
                self.contenu.append(element)

        def defile(self):
                """ Renvoie le premier element de
                la  file et l'enleve de la file """
                return self.contenu.pop(0)

        def est_vide(self):
                """ Renvoie True si la file est vide, False sinon """
                return self.contenu == []
```

Lors de la phase de tests, on se rend compte que le code suivant produit une erreur :

```
f = File()
print(f.defile())
```

3. Rectifier sur votre copie le code de la classe `File` pour que la fonction `defile` renvoie `None` lorsque la file est vide.

**Correction**

```
class File:
        ....

        .....

        def defile(self):
                """ Renvoie le premier element de
                la  file et l'enleve de la file """
                if len(self.contenu) == 0:
                        return None
                else:
                        return self.contenu.pop(0)

        .....
```

On se propose d'ordonnancer les processus avec une méthode du type tourniquet telle qu'à chaque cycle :

- si un nouveau processus est créé, il est mis dans la file d'attente ;
- ensuite, on défile un processus de la file d'attente et on l'exécute pendant un cycle ;
- si le processus exécuté n'est pas terminé, on le replace dans la file.

Par exemple, avec les processus suivants :

| Liste des processus | | |
|---|---|---|
| processus | cycle de création | durée en cycles |
| A | 2 | 3 |
| B | 1 | 4 |
| C | 4 | 3 |
| D | 0 | 5 |

On obtient le chronogramme ci-dessous :

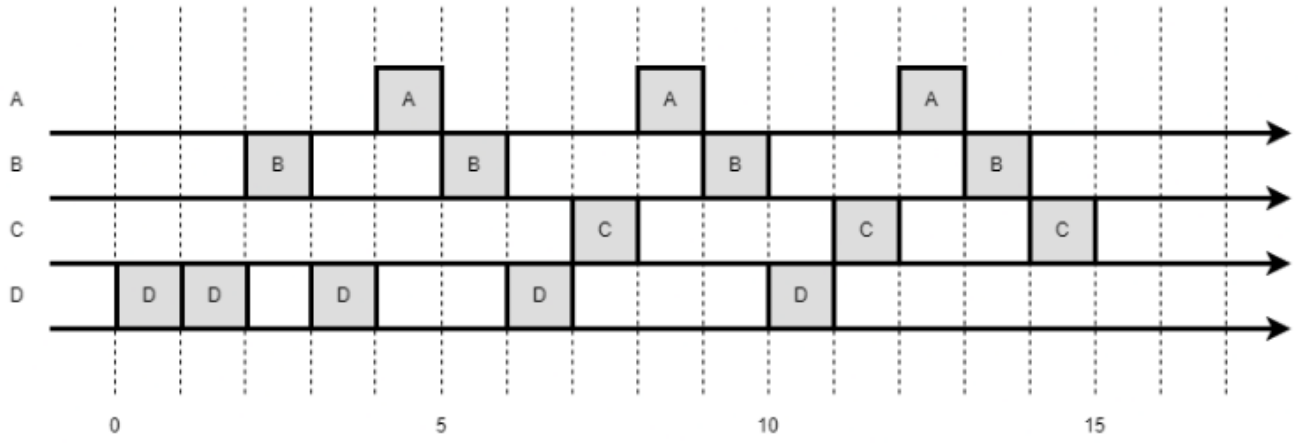


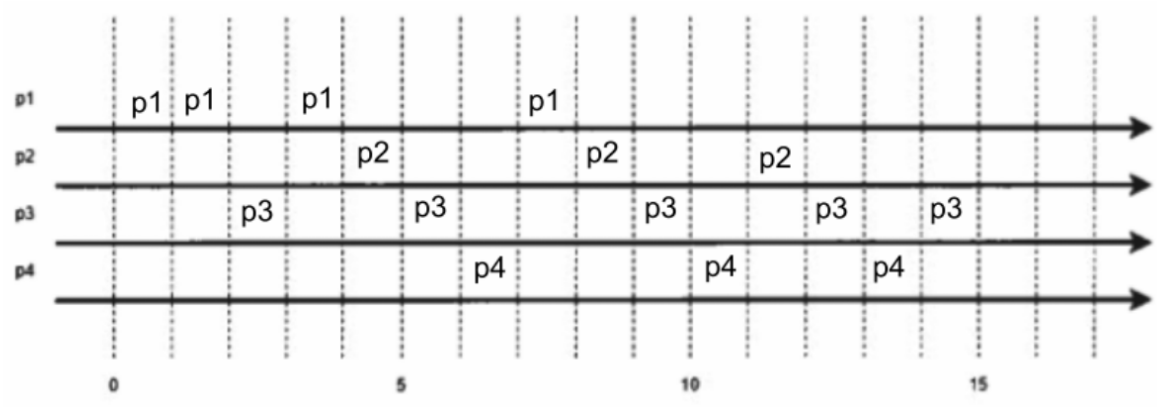

Figure 1. Chronogramme pour les processus A, B, C et D

Pour décrire les processus et le moment de leur création, on utilise le code suivant, dans lequel `depart_proc` associe à un cycle donné le processus qui sera créé à ce moment :

```
p1 = Processus("p1", 4)
p2 = Processus("p2", 3)
p3 = Processus("p3", 5)
p4 = Processus("p4", 3)
depart_proc = {0: p1, 1: p3, 2: p2, 3: p4}
```

Il s'agit d'une modélisation de la situation précédente où un seul processus peut être créé lors d'un cycle donné.

4. Recopier et compléter sur votre copie le chronogramme ci-dessous pour les processus `p1`, `p2`, `p3` et `p4`.




Pour mettre en place l'ordonnancement suivant cette méthode, on écrit la classe `Ordonnanceur` dont voici un code incomplet (l'attribut temps correspond au cycle en cours) :

```
class Ordonnanceur:

        def __init__(self):
                self.temps = 0
                self.file = File()

        def ajoute_nouveau_processus(self, proc):
                '''Ajoute un nouveau processus dans la file de
                l'ordonnanceur. '''
                self.file.enfile(proc)

        def tourniquet(self):
                '''Effectue une etape d'ordonnancement et renvoie le nom
                du processus elu.'''
                self.temps += 1
                if not self.file.est_vide():
                        proc = self.file.defile()
                        proc.execute_un_cycle()
                        if not proc.est_fini():
                                self.file.enfile(proc)
                        return proc.nom
                else:
                        return None
```

5. Compléter le code ci-dessus.

**Correction**
Voir code ci-dessus

À chaque appel de la méthode `tourniquet`, celle-ci renvoie soit le nom du processus qui a été élu, soit `None` si elle n'a pas trouvé de processus en cours.

6. Écrire un programme qui :
   - utilise les variables `p1`, `p2`, `p3`, `p4` et `depart_proc` définies précédemment ;
   - crée un ordonnanceur ;
   - ajoute un nouveau processus à l'ordonnanceur lorsque c'est le moment ;
   - affiche le processus choisi par l'ordonnanceur ;
   - s'arrête lorsqu'il n'y a plus de processus à exécuter.

**Correction**

```
ord = Ordonnanceur()
fin = False
while not fin:
        if ord.temps in depart_proc:
                ord.ajoute_nouveau_processus(depart_proc[ord.temps])
        elu = ord.tourniquet()
        if elu == None:
                fin = True
        else :
                print(elu)
```

## Exercice 2 (6 points)

Cet exercice porte sur les langages et la programmation (récursivité).

1. Voici une fonction codée en Python :

```
def f(n):
        if n == 0:
                print("Partez"!)
        else:
                print(n)
                f(n-1)
```

   (a) Qu'affiche la commande f(5) ?

   **correction**
   La commande `f(5)` affiche :

```
5
4
3
2
1
Partez!
```

   (b) Pourquoi dit-on de cette fonction qu'elle est récursive ?

   **correction**
   La fonction s'appelle elle-même

2. On a besoin pour les questions suivantes de pouvoir ajouter une chaîne de caractères `s` en préfixe à chaque chaîne de caractères de la liste `liste`. On appellera cette fonction `ajouter`. Par exemple, `ajouter("a",["b","c"])` doit retourner `["ab","ac"]`.

   (a) Recopiez le code suivant et complétez ..... sur votre copie :

   **correction**
   Voir code ci-dessous.

```
def ajouter(s, liste):
        res = []
        for m in liste:
                res  = res.append(s+m)
        return res
```

(b) Que renvoie la commande `ajouter("b", ["a","b","c"])` ?

**correction**

```
['ba', 'bb', 'bc']
```

3. On s'intéresse ici à la fonction suivante écrite en Python où `s` est une chaîne de caractères et `n` un entier naturel.

```
def produit(s, n):
        if n == 0:
                return [""]
        else:
                res = []
                for i in range(len(s)):
                        res = res + ajouter(s[i], produit(s, n-1))
                return res
```

(a) Que renvoie la commande `produit("ab", 0)` ? Le résultat est-il une liste vide ?

**correction**

```
['']
```

Il ne s'agit pas d'une liste vide, mais d'une liste contenant une chaine de caractère vide.

(b) Que renvoie la commande `produit("ab", 2)` ?

**correction**

```
['aa', 'ab', 'ba', 'bb']
```

## Exercice 3 (6 points)

Soit la fonction récursive `fib` calculant le terme de rang $n$ de la suite de Fibonacci.

```
def fib(n):
    if n<= 1:
        return n
    else:
        return fib(n-2) + fib(n-1)
```

1. Recopier sur votre copie le code de la fonction `fib` et compléter le code manquant.

2. Représenter à l'aide d'un arbre binaire la pile d'appel de cette fonction pour $n = 6$.
3. Analyser cette pile d'appel et identifier un problème majeur de cette implémentation récursive.

> On retrouve plusieurs fois le même appel appel de fonction. Cela engendre des calculs inutiles et cela augemnte considérablement la taille de la pile d'appel de fonctions, cela peut engendré un débordement de la pile.

4. Expliquer comment résoudre le problème de cette implémentation.

> Il faut utilisé une structure pour mémoriser les calculs (par exemple un tableau ou un dictionnaire). Dès que la calcul d'un terme de la suite de fibonacc est terminé, on le stocke dans la structure de donnée et lors des appels récursif suivants avec ce même paramètres, on peut repredre directement la valeur calcullée sans appelé la fonction récursive.